



LECTURE 1

C++ Review + ADT Design

CS24 · Fall 2026 · Diba

CS24 Course Map

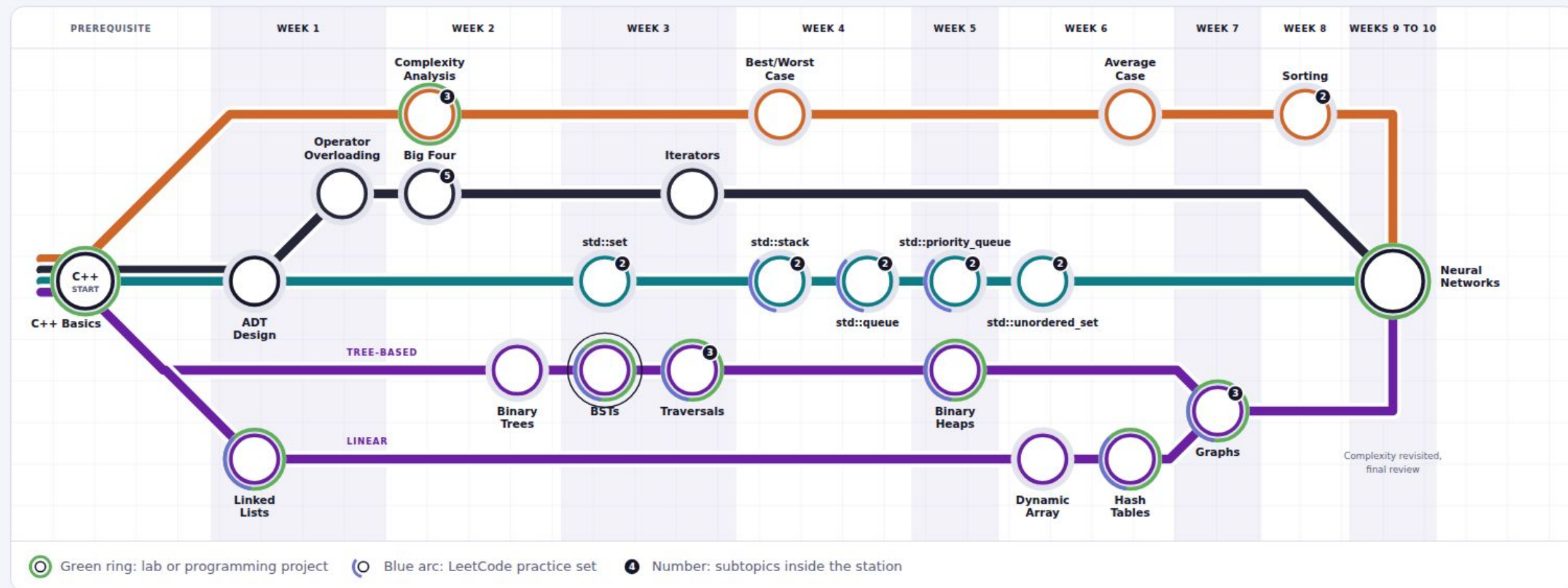
Click a station for its subtopics, reading, and coursework.

C++

ADTs

Theory

Data Structures



BEFORE WE START

Logistics Overview

- Grading: · Practice (Leetcode + Caliber) 10% · Labs + PAs: 20%
 - Quizzes 20% · Mock Interview 15% · Final 35%
- 70% of your grade is paper exams and live coding interviews: no AI there
- Textbook: Savitch. Pre-reading is listed on the course site before each lecture
- Mock interviews run on Caliber: Intro (2m) → Whiteboard (10m) → Coding (13m) → Feedback (5m)
- Read the syllabus on the course website: <https://ucsb-cs24.github.io/f26/>

Quick gauge before we start

How comfortable are you with pointers and dynamic memory in C++ right now?

- A — Never used them
- B — Seen them, haven't written much
- C — Comfortable with the basics
- D — Very comfortable



<https://join.iclicker.com/BCRL>

WHY THIS COURSE

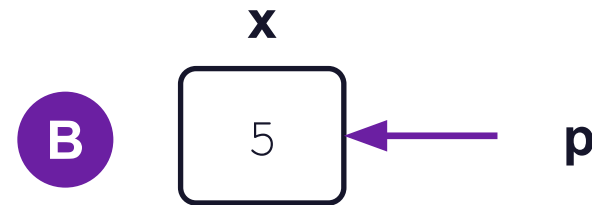
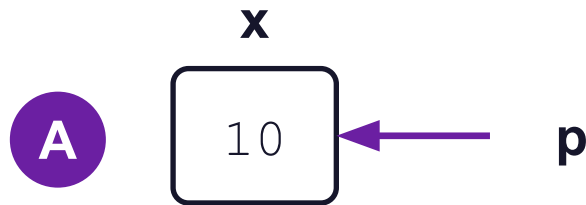
Why code data structures?

- AI can write a correct linked-list reversal, why should you do it?
- Coding a problem is an expression of clear understanding, ability to build from first principles.
- The mock interview is where you will practice expressing and communicating your solutions.
- That's the same skill a real technical interview tests and the same skill behind every “why is my code slow / leaking memory / crashing” moment later

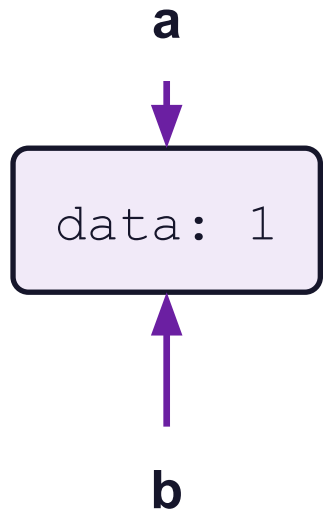
70% OF YOUR GRADE: PAPER EXAMS + LIVE INTERVIEWS, NO AI

Question 1

```
int x = 5;
int* p = &x;
*p = 10;
// which diagram matches memory now?
```



Question 2

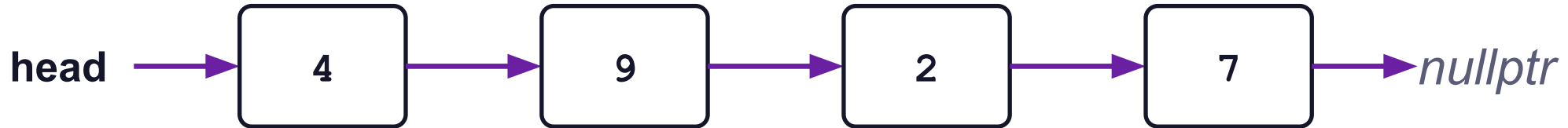


```
Node* a = new Node(1);  
Node* b = a;  
b->data = 99;  
  
// what is a->data now?
```

- A — 1
- B — 99
- C — a crash
- D — undefined

NEW STRUCTURE

Anatomy of a linked list



```
struct Node {  
    int data;  
    Node* next;  
};
```

- head is the only pointer we store
- Everything else: follow →next until nullptr

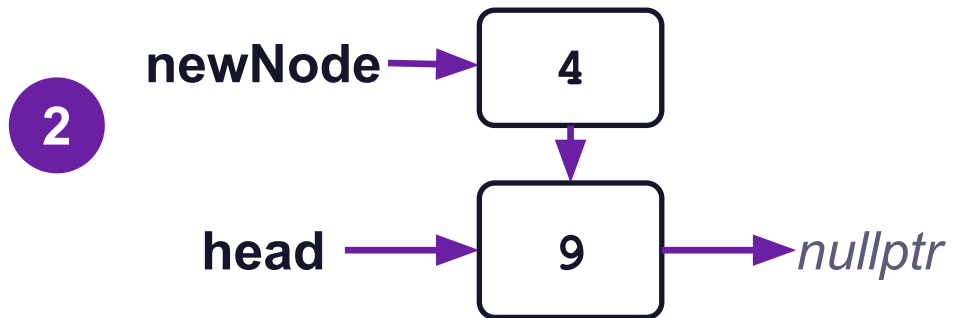
How would you add a node?

Could you write the code to insert a new node — at the head? At the tail?

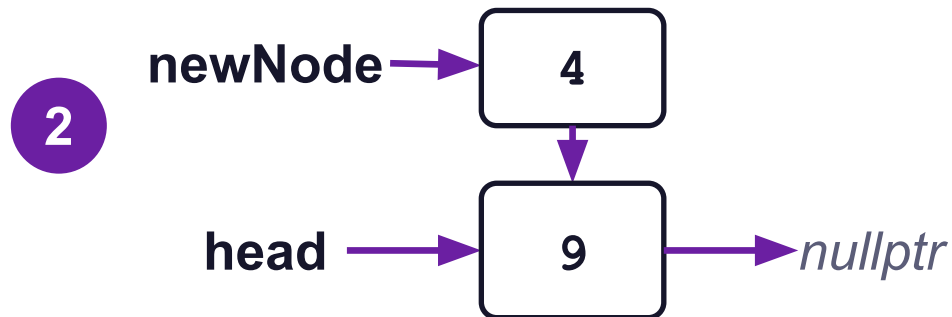
- A. Neither yet, show me first
- B. I could do the head
- C. I could do the tail
- D. I could do both

WALKTHROUGH

Adding a node to the head



Which code adds a node to the head?



```
Node* newNode = new Node(value);
```

A) `newNode->next = head; head = newNode;`

B) `head = newNode; newNode->next = head;`

C) `newNode->next = newNode; head = newNode;`

D) `head->next = newNode;`

THE PROBLEM

Reverse a linked list

```
Given the head of a singly linked list,  
reverse the list, and return the new head.
```

Example:

```
Input:    1 -> 2 -> 3 -> 4 -> 5 -> nullptr
```

```
Output:   5 -> 4 -> 3 -> 2 -> 1 -> nullptr
```

A classic interview question, the same shape you'll see on LeetCode.

Reverse a Linked List

Mapped to the mock-interview rubric

- ☐ Restate + edge case
- ☐ Test case: in → out
- ☐ Recursive or iterative?
- ☐ Derive it, think aloud
- ☐ Name the base case
- ☐ Name the subproblem(s)
- ☐ Combine into the answer
- ☒ Complexity + adapt

(whiteboard your solution)

CONCEPT

What is an Abstract Data Type (ADT)?

- Need a collection that grows and shrinks?
- Use C++ ADTs : **std::vector** or **std::list**
- An ADT is defined by what it does (the interface), hiding the implementation details
- Goal: turn our chain of nodes into a first-class ADT like `std::list`. That's `CustomList`

CustomList, our ADT:

- `push_back(value)`
- `print()`

(hides: head, tail, Node internals)

Critique customList ADT in main.cpp

1A

Find at least 3 problems with the flawed CustomList in main.cpp.

- Open main.cpp: this is the version we will NOT keep
- Work with the person next to you, 9 minutes
- Look for: memory issues, slow operations, missing safety checks
- We'll fix it together right after. Write the constructor and push_back, live

Constructor + push_back

```
CustomList::CustomList() {  
    head = nullptr;  
    tail = nullptr;  
}  
  
void CustomList::push_back(int val) {  
    // why was push_back slow?  
    // ... build it with tail  
}
```

- Ask before revealing: why is main.cpp's push_back slow?
- Answer lives in the tail pointer we're about to add

ACTIVITY 1C · ON YOUR OWN

Implement print()

1C

Write CustomList::print() yourself.

```
void CustomList::print() const {  
    // your code here  
}
```

ONE MORE THING

```
CustomList list2 = list1;  
  
// ... what just happened?
```

We'll draw exactly what this does to memory — next lecture.

SETUP FOR NEXT TIME

Meet Complex

```
class Complex {  
public:  
    Complex(double r, double i);  
    double getReal() const;  
    double getImag() const;  
private:  
    double real, imag;  
};
```

- Next lecture: operator overloading on Complex class, not CustomList
- CustomList's operators are your lab
- Complex shows the mechanics first

BEFORE NEXT TIME

Read, answer, draw, and code

- Read: Savitch 4.3–4.4 (operator overloading) and 5.1, 5.3 (constructors/destructors, Rule of Three)
- Write an answer: which function does `cout << x` actually call, and why?
- Write an answer: does `Complex` need a destructor? Why or why not?
- Draw and predict: the diagram for `CustomList list2 = list1`? What problems could arise as a result of this operation.
- Homework: rewrite `reverse()` iteratively (prev/curr, a while loop)